AESTHETIXAI · SURGENT OPERATING SYSTEM

The Complete Project Documentation & 31-Agent Specification

An autonomous AI workforce for plastic surgery & aesthetic medicine practices — architecture, current build status, the doctor and admin panels, and a full specification of all 31 specialist agents for team assignment.

FASTAPI + POSTGRES | REACT + TYPESCRIPT | OPENAI · MISTRAL | DEEPGRAM · GROQ | VPS DEPLOYMENT

Written by: Furqan Raza

Project team: Wahaj & Muneeb

Repository: github.com/aiaceonesolutionscom/Surgent_Operating_System

Date: August 22, 2026

CONTENTS

Table of Contents

یہ دستاویز پروجیکٹ کی موجودہ حالت، مکمل آرکیٹیکچر، اور تمام 31 ایجنٹس کی تفصیل کور کرتی ہے — تاکہ ٹیم ممبرز (وہاج اور منیب) کو اپنا اپنا ایجنٹ اسائن کر کے کام شروع کروایا جا سکے۔

SECTION 01

Executive Summary

Aesthetixai (Surgent Operating System) is being built as an autonomous AI workforce for plastic surgery and aesthetic medicine practices — 31 specialist agents that together cover the full patient journey, from the very first call or Instagram DM through to full post-surgical recovery. The idea is not one generic chatbot; it's 31 narrow, purpose-built agents, each owning one job end to end, so a practice can turn agents on incrementally and each one has a clear, measurable job to do.

The repository is a clean monorepo split into a **frontend** (Vite + React + TypeScript marketing site and future practice dashboard) and a **backend** (FastAPI + Postgres API that will hold all 31 agents' real logic). The codebase is deliberately organized *"one agent, one place"* — every agent gets an identically-named folder in both stacks, so a team member assigned to an agent never has to touch anyone else's files.

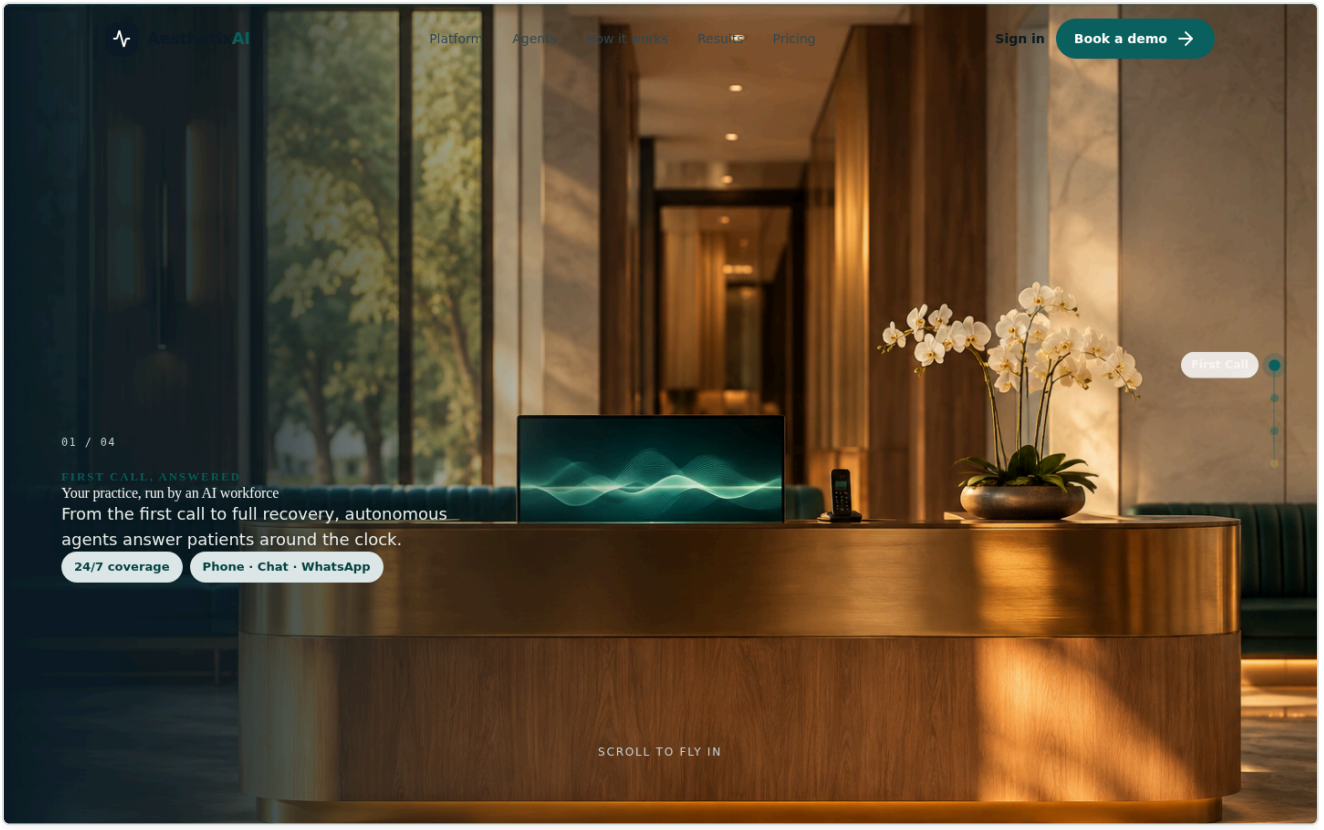
AREA	HONEST CURRENT STATE
Public marketing website	Largely built and polished — cinematic hero, agent catalog (all 31 listed), channels page, pricing page. This is the most finished part of the project today.
Backend agent logic	All 31 agents are scaffolded (router → controller → service for each). Only **1 of 31** — the AI Receptionist — has real logic today (LLM + Twilio call handling). Every other agent's service is a stub that just returns `{"status": "active"}`.
Doctor panel & admin panel	Not built yet. Four AI-generated UI mockups exist as design direction (voice receptionist live view, executive dashboard, patient timeline, advanced analytics) — kept explicitly as reference only, not wired to real data.
Database	Postgres schema is solidly underway — 13 core models (Practice, Patient, User, Appointment, Procedure, Invoice, Conversation, Message, PatientPhoto, RecoveryJournal, AgentConfig, AgentLog, Subscription) with Alembic migrations.
Deployment	Dockerfiles exist for both frontend (nginx, static) and backend (uvicorn). Not yet deployed to a live VPS.

This document exists so that **Furqan** can hand each of the 31 agents to **Wahaj** and **Muneeb** as a self-contained, well-specified unit of work — what the agent does, what real-world problem it solves for a clinic, which AI/API stack it needs (OpenAI and Mistral for language, Deepgram and Groq specifically for the agents that handle live voice), and roughly how valuable it is to the business — so build priority is obvious and no one has to guess.

SECTION 02

What Aesthetixai Is — Product Vision

The pitch on the live marketing site says it best: *"Your practice, run by an AI workforce."* Instead of one chatbot bolted onto a clinic's website, Aesthetixai is a set of 31 specialist AI agents, each responsible for exactly one job a real front-desk staffer, nurse coordinator, scheduler, or billing clerk would otherwise do — available 24/7, across phone, chat, WhatsApp, Instagram, and every other channel a practice's patients already use.

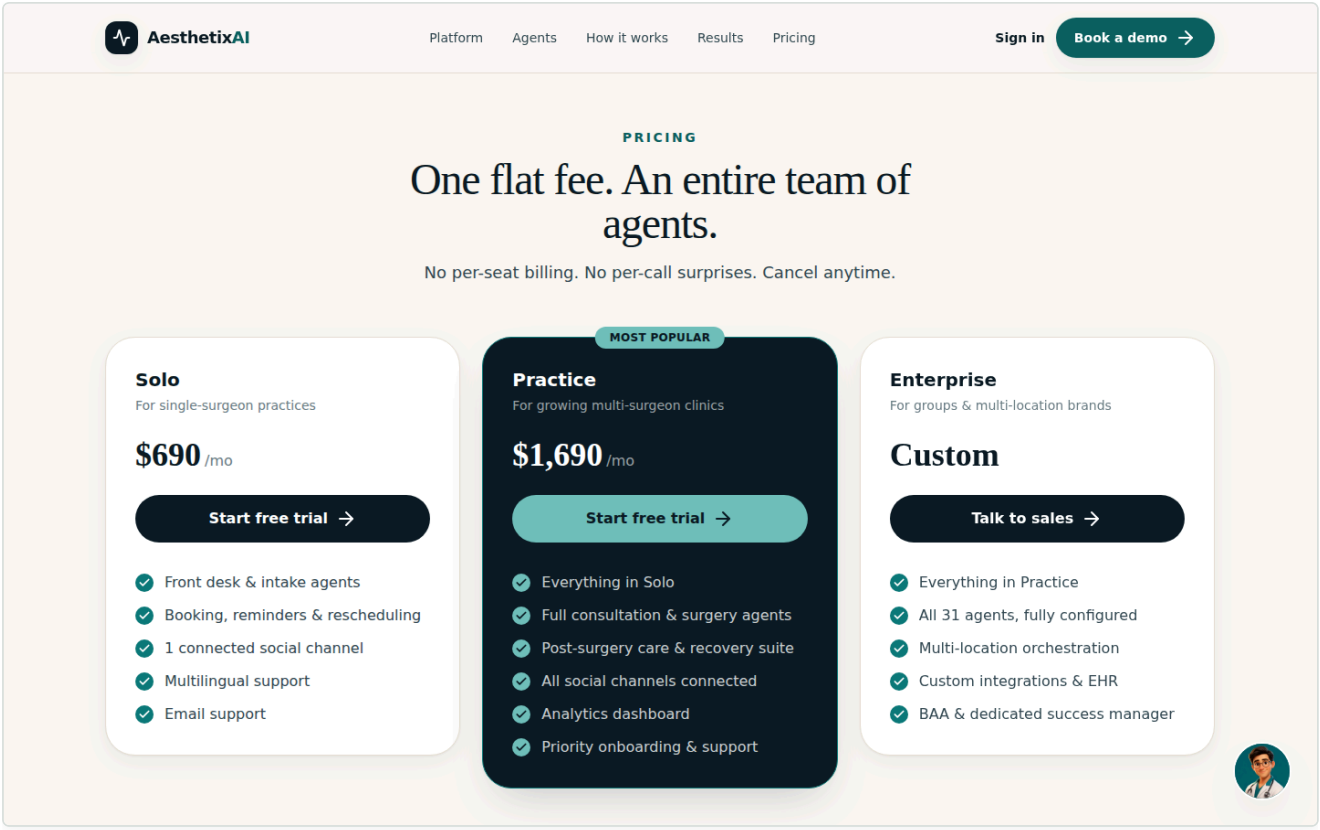


Live preview — the current marketing homepage (localhost, this session), showing the cinematic "First Call, Answered" hero sequence.

The five stages of the patient journey the agents cover

STAGE	WHAT HAPPENS HERE	# AGENTS
Front Desk & Intake	The very first contact — a call, a DM, a web form — gets answered instantly and turned into a booked consultation.	5
Consultation & Screening	Pre-consult qualification: history, photos, risk factors, and procedure fit are gathered before the surgeon spends a minute of their own time.	7
Surgery Management	Once booked, the surgical calendar, OR, equipment and implants are coordinated so nothing delays the day of surgery.	6
Post-Surgery Care	Recovery is monitored, medication and wound-care guidance is delivered, and emergencies are triaged instantly.	6
Business & Operations	Quoting, payments, insurance, reviews, marketing follow-up and the analytics that tell the owner what's working.	7

Today's live product pricing (already coded into the marketing site's Pricing section) packages these agents into three tiers:



Live preview — current pricing section (Solo $690/mo, Practice $1,690/mo, Enterprise custom).

SECTION 03

Architecture & Codebase Structure

The repository is a monorepo with three top-level folders: `frontend/` (the public site + future dashboard), `backend/` (the FastAPI agent platform), and `design-references/` (AI-generated UI mockups, kept for direction only — not built, not part of either running app).

The core design decision: "one agent, one place"

Both stacks are deliberately organized so that every one of the 31 agents gets its own identically-named folder in the backend *and* the frontend. This is the single most important thing for team members to understand before picking up an agent: **you should never need to touch another agent's files, or shared layout/UI code, to build yours out.**

Backend layout — FastAPI, layer-first

```
backend/src/
├── router/agents/<name>_agent/<name>_agent_router.py       HTTP routes + auth
├── controller/agents/<name>_agent/<name>_agent_controllers.py  adapts request → service call
├── services/agents/<name>_agent/<name>_agent_services.py       the REAL logic goes here
├── models/           SQLAlchemy models — Practice, Patient, Appointment, Procedure...
├── schemas/          Pydantic request/response contracts
├── server/           auth dependency (Clerk), middleware, exception handling
└── services/         shared, non-agent services: llm, twilio, whatsapp, instagram,
                       cloudinary, email, payment, calendar, clerk, ehr, storage
```

The call chain for every one of the 31 agents is the same: **Router → Controller → Service.** The router defines the HTTP route and who's allowed to call it; the controller adapts the incoming request into a service call; the service holds the actual business logic and is where shared integrations (LLM, Twilio, WhatsApp...) get used. In practice, **filling in `..._services.py` is almost always the only file a developer needs to touch** — the router/controller shape rarely needs to change once an agent exists.

Frontend layout — React 18 + TypeScript + Tailwind

```
frontend/src/
├── components/   layout, hero (cinematic scroll video), sections (pricing, testimonials...),
│                 features (AgentSuite catalog, Omnichannel), ui (Button, Container)
├── pages/        HomePage, AgentsPage, AgentDetailPage, ChannelsPage
├── data/agents/  ONE FILE PER AGENT — <slug>.ts — name/description/icon/category
├── api/          typed fetch wrapper (getHealth() is the one real call today)
└── app/          FUTURE authenticated practice dashboard — auth/, dashboard/agents/<name>_agent/,
                  dashboard/profile/ — scaffolded, not wired up (no frontend auth yet)
```

Right now, `data/agents/<slug>.ts` is the entire frontend footprint of most agents — a small file with name, one-line description, icon, and category, rendered through one shared `AgentDetailPage.tsx` template rather than 31 hand-built pages. The real, per-agent authenticated dashboard screens (`app/dashboard/agents/<name>_agent/`) exist as empty placeholder files today, waiting for both the backend logic and frontend auth to exist.

> **Rule of thumb for whoever is assigned an agent** — say, `surgery_scheduling`: edit `backend/src/services/agents/surgery_scheduling_agent/surgery_scheduling_agent_services.py` for the real logic, its controller/router only if adding a new operation/endpoint, and `frontend/src/data/agents/surgery_scheduling.ts` for any copy changes. Nothing else. If you find yourself needing to edit another agent's folder or a shared component, that's a sign the change belongs in a shared service instead.

SECTION 04

Technology & AI Stack

The backend today authenticates against OpenAI only. Going forward, per Furqan's direction, the AI layer is split by cost and by whether an agent has to hold a real-time voice conversation:

PROVIDER	ROLE IN THIS PROJECT	USED BY
OpenAI (GPT-4o)	Primary reasoning model — complex conversations, medical-context reasoning, vision (photo analysis), anything where quality/accuracy matters more than raw cost.	Receptionist, AI Consultation, Photo Analysis, Risk Assessment, Emergency Triage, Surgery Scheduling, OR Scheduler, Surgical Documentation, Lead Nurturing, and as the reasoning layer behind most other agents.
Mistral	Cost-efficient LLM for high-volume, lower-stakes text — reminders, templated messages, simple intent parsing, translation drafts, follow-up copy. Keeps OpenAI spend down on the agents that fire thousands of times a day.	Appointment Reminder, Reschedule & Cancellation, Pre-Surgery Prep, Medication Reminder, Marketing Follow-up, Cost Estimation drafting, Equipment Checklist, Implant Inventory.
Deepgram	Real-time speech-to-text (STT) — turns a live phone call into text the LLM can reason over, streaming, sub-second.	Every agent that answers a live call: Receptionist, Emergency Triage, Multilingual Translation (live calls), optionally Surgical Documentation (voice dictation).
Groq	Ultra-low-latency LLM inference (LPU hardware) — used specifically inside the voice pipeline so the AI's spoken reply doesn't lag behind the patient on the call. Text-chat agents don't need this; a 1–2 second delay is invisible in chat but is very noticeable on a live phone call.	Same voice-first agents as Deepgram: Receptionist, Emergency Triage, Multilingual Translation.
Twilio	Telephony (inbound/outbound calls, SMS) and the TwiML bridge that connects a live call to the AI pipeline.	Receptionist, Appointment Reminder (voice reminders), Emergency Triage (staff escalation calls/SMS).
WhatsApp Business API & Instagram/Facebook (Meta)	The social/chat channels patients actually use — this is the "social media integration" layer.	Lead Nurturing, Marketing Follow-up, Reschedule & Cancellation, Recovery Follow-up, Patient Feedback, and the Receptionist for chat.
Stripe	Payments, deposits, invoicing, subscription billing for the practice's own SaaS plan.	Payment & Invoice, Cost Estimation.
Cloudinary	Shared file/image storage — any agent that saves a photo or document uses this, it isn't owned by one agent.	Photo Analysis, Healing Monitoring, Surgical Documentation, Medical History Intake (documents).
Clerk	Authentication for the practice-side app (staff/doctor logins). Backend auth dependency is wired; frontend sign-in flow is not yet built.	All authenticated (non-public) API routes.
SendGrid	Transactional email — confirmations, receipts, reports.	Appointment Booking, Payment & Invoice, Patient Feedback, Marketing Follow-up.
Google Calendar	Two-way calendar sync for surgeons and consult slots.	Appointment Booking, Surgeon Calendar, Surgery Scheduling.
Postgres + SQLAlchemy 2.0 (async) + Alembic	The single source of truth for every practice, patient, appointment, and agent log — chosen specifically for VPS self-hosting per Furqan's direction.	Every agent.

Celery + Redis	Background job queue — for anything that shouldn't block an HTTP response (reminder scheduling, bulk follow-ups, report generation).	Appointment Reminder, Recovery Follow-up, Marketing Follow-up, Analytics Dashboard.

> **Why the OpenAI/Mistral split matters financially:** a practice's AI Receptionist alone can handle hundreds of calls a month. Running every one of those through GPT-4o is far more expensive than it needs to be for the ~80% of turns that are simple ("what's your address", "can I reschedule"). Routing simple, templated, high-volume traffic to Mistral and reserving GPT-4o for genuinely complex reasoning (medical history, risk assessment, live call reasoning) is a direct cost-control lever — this should be treated as a real engineering decision per agent, not an afterthought.

SECTION 05

Current Build Status — What % Is Ready

This is an honest, code-level read of the repository as of August 22, 2026 — not a guess. Each row reflects what actually exists in the codebase today versus what is scaffolded or still missing.

AREA	EST. %	EVIDENCE
Public marketing website	~85%	Hero, navbar/footer, agent catalog (all 31), agent detail template, channels page, pricing section, testimonials — all live and rendering. Remaining: real "Book a demo" / "Sign in" flows are not yet wired to a backend.
Practice / doctor dashboard (frontend)	~5%	`app/dashboard/agents/<name>/` and `app/auth/` exist only as placeholder files/README stubs. No authenticated screens render real data yet.
Admin panel	~0% (built) / design-ready	No admin app exists in either stack yet. Direction exists via the Executive Dashboard and Advanced Analytics mockups (Section 06).
Backend shared infrastructure	~55%	Auth dependency, DB connection/session, exception handling, and 12 shared service modules (LLM, Twilio, WhatsApp, Instagram, Cloudinary, Email, Payment, Calendar, Clerk, EHR, Storage) all exist as real files. LLM and Twilio services have working code; several others (EHR, Storage) are thinner.
Backend agent business logic (31 agents)	~3% (1 of 31)	Every agent has a router + controller + service file (100% scaffolded). Only `receptionist_agent_services.py` has real logic (LLM call + Twilio TwiML). The remaining 30 return a static `{"status": "active"}` stub.
Database schema	~55%	13 core SQLAlchemy models cover practices, users, patients, appointments, procedures, invoices, conversations/messages, patient photos, recovery journals, agent config/logs, and subscriptions. Agent-specific tables not yet modeled: OR/equipment schedules, implant stock, insurance claims, surgical documentation records, medication adherence logs.
Social & channel integrations	~30%	Twilio and WhatsApp service files exist and are structured; Instagram service file exists but is thin. None are yet driving a built-out agent besides the receptionist's Twilio path.
Deployment / DevOps	~50% (ready, not live)	`frontend/Dockerfile` (nginx + SPA fallback) and `backend/Dockerfile` both exist and are documented. No evidence yet of an actual VPS deployment.

Rough overall completion

Weighting the areas above by how much of the eventual product they represent (agent logic and the two dashboards being the largest remaining chunks of work), the project is realistically at:

~18–20%

of the full product built

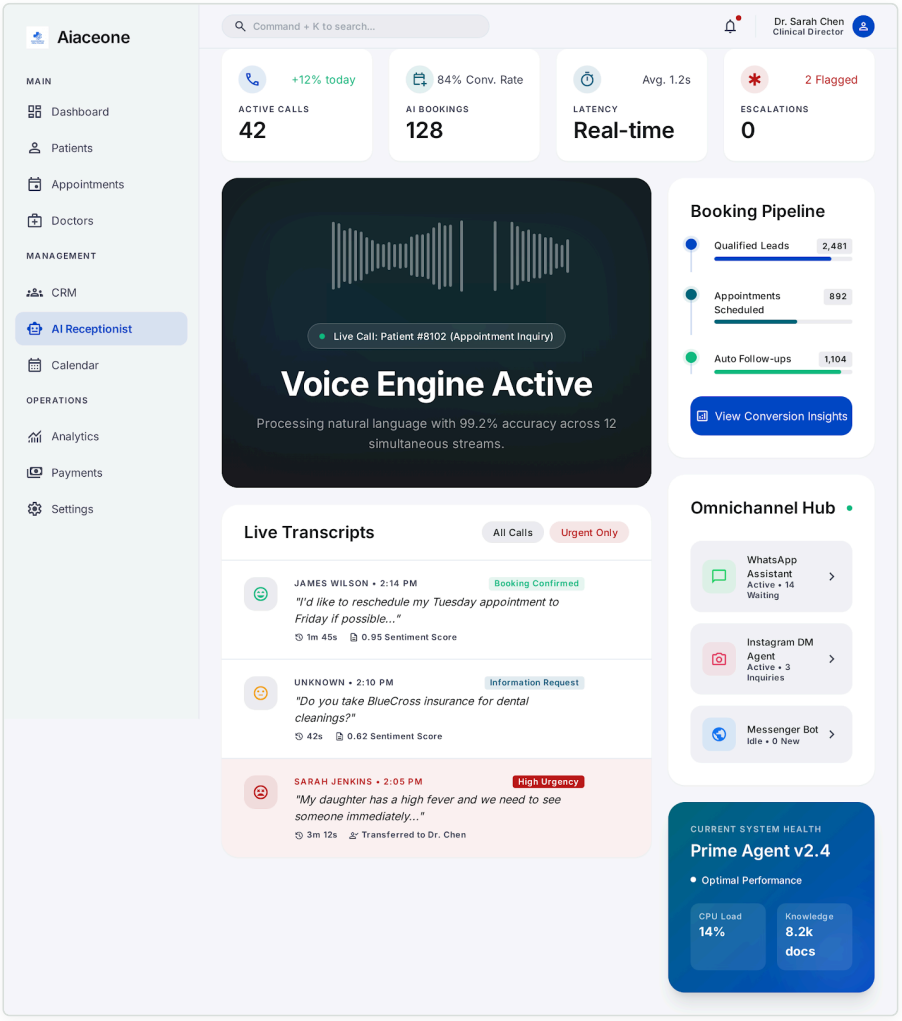
> **Read this the right way:** the foundation is exactly where you'd want it before scaling a team on it — the hard architectural decisions (folder structure, auth pattern, DB schema, one working reference agent to copy from) are made. What's left is mostly repetitive, well-scoped work: filling in 30 more `_services.py` files against a pattern that already exists, plus building the two dashboards once a handful of agents have real data to show.

SECTION 06

Doctor Panel & Admin Panel

Neither panel is built yet in code — both live today only as AI-generated design mockups in `design-references/For.UI/`, explicitly kept as direction, not implementation. They are, however, an excellent and very concrete blueprint for what the `frontend/src/app/` dashboard needs to become once agents have real data to show. Four reference screens exist:

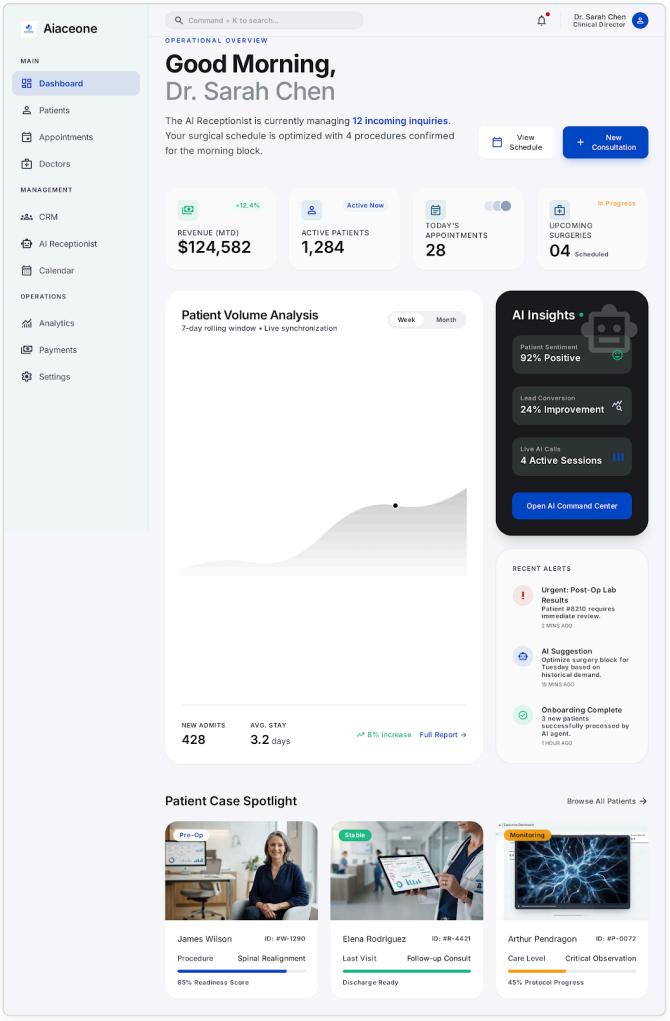
1 — AI Receptionist live view (doctor/staff panel)



Design reference — live call monitoring: active calls, AI booking conversion rate, latency, escalations, a live waveform + transcript feed, booking pipeline, and an omnichannel hub (WhatsApp/Instagram/Messenger) sidebar.

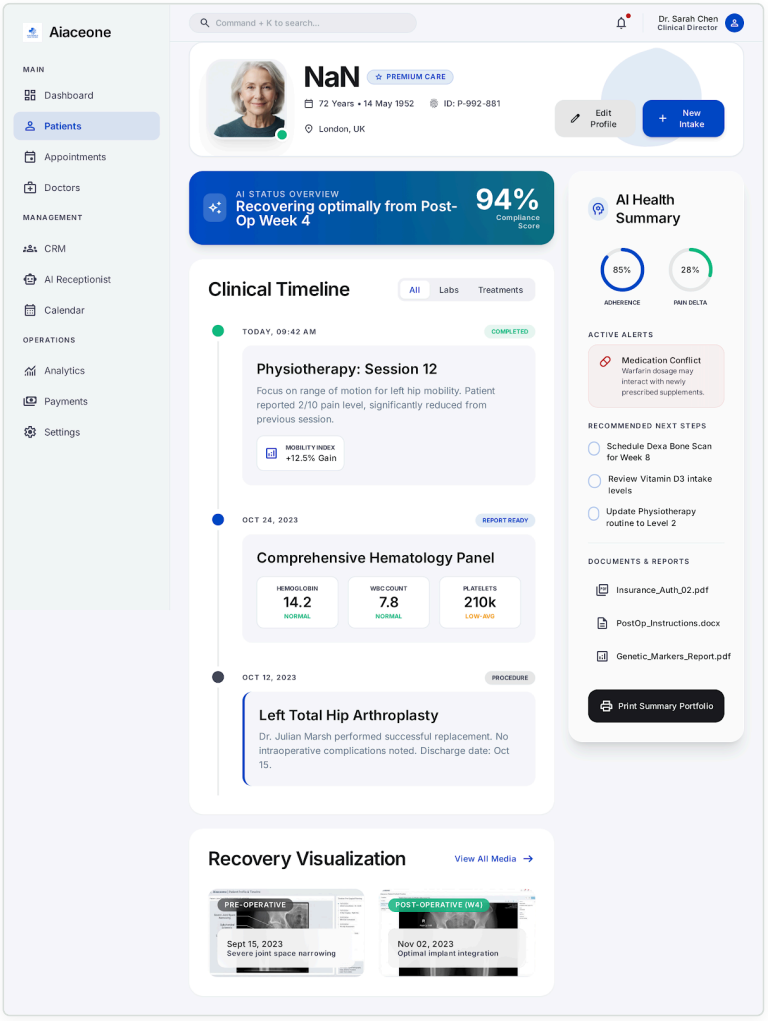
This is the natural home screen for the Receptionist and Lead Nurturing agents once built — it's the one place staff would watch to see what the AI is doing on the phones and in DMs, right now, in real time.

2 — Executive / clinical dashboard (doctor panel home)



Design reference — a doctor's morning view: revenue MTD, active patients, today's appointments, upcoming surgeries, an AI insights panel (sentiment, lead conversion, live call count), recent alerts, and a patient case spotlight.

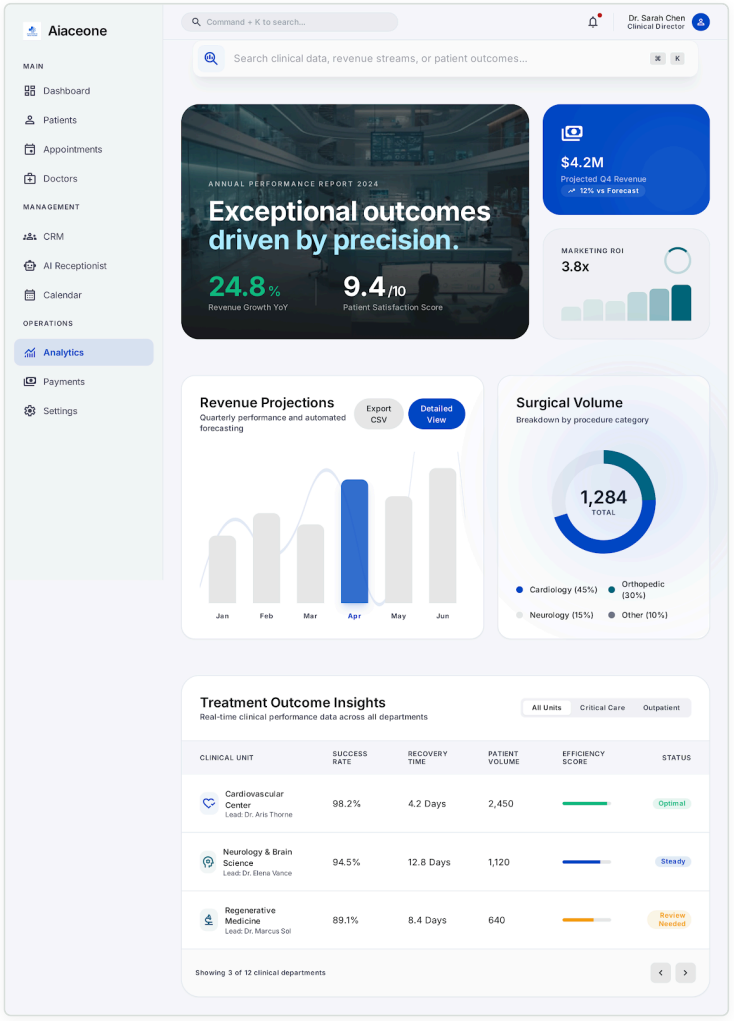
3 — Patient profile & clinical timeline



Design reference — the single-patient view: AI status overview (recovery %, compliance score), a clinical timeline (physio sessions, lab panels, procedures), an AI health summary with active alerts (e.g. medication conflicts), and recommended next steps.

This screen is where **Healing Monitoring**, **Recovery Follow-up**, **Medical History Intake**, and **Surgical Documentation** all converge — it's the clinical record view a surgeon would actually open between patients.

4 — Advanced analytics (admin panel)



Design reference — practice-wide admin view: revenue growth, patient satisfaction, marketing ROI, revenue projections, surgical volume by category, and a treatment-outcomes table across clinical units.

This is the natural home for the **Analytics Dashboard** agent, and functionally the admin/owner-level panel Furqan described — the place a practice owner (not necessarily a clinician) checks to see whether the AI workforce is actually growing the business.

> **Build note:** both panels need Clerk sign-in wired on the frontend before any of this can render real data — today `app/auth/` is an empty placeholder. The realistic sequencing is: (1) wire frontend auth → (2) build out 4–6 of the highest-value agents with real backend logic → (3) build the doctor panel screens against those agents' real data → (4) build the admin/analytics panel once enough agents are producing real numbers to aggregate.

SECTION 07

Database & VPS Deployment Plan

Database — Postgres

Postgres was chosen specifically because this is going to be self-hosted on a VPS rather than run on a managed serverless database — it's the right call for cost control and for keeping patient data on infrastructure the practice/agency fully controls. Access is async throughout (SQLAlchemy 2.0 async + asyncpg), with Alembic managing schema migrations so the schema can evolve safely as each of the 30 remaining agents gets built out and needs its own tables.

MODEL (EXISTS TODAY)	PURPOSE
Practice	The clinic/tenant itself — name, contact info, timezone, settings.
User	Staff/doctor accounts, tied to Clerk auth.
Patient	Core patient record — contact info, DOB, medical history (JSONB), consent status.
Appointment	Bookings — consult or surgery, tied to patient + practice.
Procedure	The catalog of procedures a practice offers.
Invoice	Billing records, tied to Stripe.
Conversation / Message	Every AI ↔ patient interaction across every channel — the backbone almost every agent will read/write.
PatientPhoto	Uploaded photos (screening, recovery), stored via Cloudinary, referenced here.
RecoveryJournal	Post-op check-in entries.
AgentConfig / AgentLog	Per-practice agent settings and an audit trail of what every agent did — important for both debugging and clinical/legal accountability.
Subscription	The practice's own billing plan (Solo / Practice / Enterprise).

Still needed as agents get built out: OR/equipment schedule tables, implant inventory + reorder tables, insurance verification/claims records, structured surgical documentation records, and medication-adherence logs. Each of these should be added by whoever builds that specific agent, following the existing model conventions.

VPS sizing

STAGE	RECOMMENDED SPEC	WHY
Today / early build	2 vCPU / 2 GB RAM	The frontend is essentially free at runtime (static files behind nginx, no Node process). The backend is light — only the Receptionist constructs real SDK clients today; every other agent's service has no dependencies yet. Uvicorn with 1–2 workers is enough — FastAPI is async and doesn't need more processes for I/O-bound work at this stage.
Once several agents are real	4 GB+ RAM, add Celery/Redis workers	As more agents construct real SDK clients (OpenAI, Twilio, WhatsApp, Cloudinary, Stripe), memory scales with actual traffic — provided those clients are built *lazily* (on first use, not at app boot in a service's `__init__`, which currently runs at every app start).

Deployment shape: `frontend/Dockerfile` builds the static site and serves it via nginx (with SPA fallback so routes like `/agents/:slug` work on a direct load). `backend/Dockerfile` runs the FastAPI API directly via uvicorn. Both are ready to deploy behind a reverse proxy (e.g. Caddy/nginx with TLS) on a single VPS to start, with Postgres and Redis as sibling containers or managed instances on the same box.

SECTION 08

The 31 Agents — Full Specification

Every agent below follows the same template: what it actually does, the real business problem it solves for a practice, the AI/API stack it needs, whether it handles live voice (and therefore needs Deepgram + Groq), its estimated standalone value to a practice, and how complex it is to build. Value stars (★) reflect business impact, not build effort — a ★★★★★ agent is one that materially drives revenue or prevents a costly failure; complexity is rated separately.

> نوٹ: ہر ایجنٹ کی "قیمت/ویلیو" ایک اندازہ ہے — یہ اس بات کا اشارہ ہے کہ وہ کلینک کے لیے کتنا مالیت پیدا کرتا ہے، حتمی پرائسنگ نہیں۔

Front Desk & Intake — 5 agents — the first contact with every patient

1. AI Receptionist

`receptionist`

DOES Answers every inbound call, WhatsApp message, and web chat 24/7 in a warm, human-like voice; greets the patient, understands why they're calling, and either books them, answers FAQs, or hands off to staff. This is the one agent with real logic in the repo today.

PROBLEM SOLVED Practices lose a large share of inbound calls after hours or while staff are with patients — every missed call is a potential missed booking worth thousands of dollars.

STACK OpenAI GPT-4o | Deepgram (STT) | Groq (low-latency) | Twilio | WhatsApp

VALUE ★★★★★ Flagship — equivalent to a full-time receptionist, but 24/7 | **COMPLEXITY** High | **STATUS** Built (real logic)

2. Appointment Booking

`appointment_booking`

DOES Takes booking intent from the receptionist or the website and finds/holds a real calendar slot, confirms it by SMS/WhatsApp/email, and syncs with the surgeon's calendar.

PROBLEM SOLVED Manual back-and-forth to find an open slot loses leads to competitors and causes double-booking errors.

STACK OpenAI / Mistral | Google Calendar | Twilio / WhatsApp / SendGrid

VALUE ★★★★★ Flagship — the actual conversion moment | **COMPLEXITY** Medium–High | **STATUS** Scaffolded only

3. Reschedule & Cancellation

`reschedule_cancellation`

DOES Lets patients reschedule or cancel through any channel instantly, and automatically offers the freed slot to the next waitlisted patient.

PROBLEM SOLVED Manual reschedules eat front-desk time and leave calendar gaps that never get refilled.

STACK Mistral | Google Calendar | WhatsApp / SMS

VALUE ★★★★ High | **COMPLEXITY** Medium | **STATUS** Scaffolded only

4. Appointment Reminder

`appointment_reminder`

DOES Sends smart, timed reminders with confirm/reschedule links, escalating to an outbound call for high-value procedures at risk of a no-show.

PROBLEM SOLVED No-shows waste already-committed OR and consult time — a direct, recurring revenue loss.

STACK Mistral | Twilio / WhatsApp / SendGrid | Deepgram + Groq (voice reminder calls)

VALUE ★★★★ High | **COMPLEXITY** Low–Medium | **STATUS** Scaffolded only

5. Multilingual Translation

`multilingual_translation`

DOES Detects the patient's language automatically and responds in it across chat and live calls.

PROBLEM SOLVED Practices lose international and non-native-English-speaking patients who can't comfortably communicate with an English-only front desk.

STACK OpenAI / Mistral | Deepgram + Groq (live calls)

VALUE ★★★ Medium — niche, but high lifetime value per patient | **COMPLEXITY** Medium | **STATUS** Scaffolded only

Consultation & Screening — 7 agents — qualifying and preparing every patient before the surgeon's time is spent

6. AI Consultation

`ai_consultation`

DOES Runs a structured conversational pre-consult — goals, concerns, budget — and produces a clean summary for the surgeon before the real consult begins.

PROBLEM SOLVED Surgeons burn expensive, scarce time re-asking basic questions; unqualified consults waste that time entirely.

STACK OpenAI GPT-4o | Mistral (high-volume fallback)

VALUE ★★★★★ Flagship — protects the surgeon's time, the scarcest resource | **COMPLEXITY** High | **STATUS** Scaffolded only

7. Photo Analysis (Screening)

`photo_analysis`

DOES Screening-only review of patient-submitted photos to flag areas of interest for the surgeon — explicitly never a diagnosis, always framed as a conversation starter.

PROBLEM SOLVED Patients want quick feedback before committing to a consult; surgeons want photos pre-sorted and organized, not a folder of unlabeled images.

STACK OpenAI GPT-4o Vision | Cloudinary

VALUE ★★★★ High | **COMPLEXITY** High — medical-image handling is liability-sensitive; needs clear disclaimers baked into every response | **STATUS** Scaffolded only

8. Video Consultation

`video_consultation`

DOES Schedules and hosts secure virtual visits, with automated pre-visit reminders and an AI-generated post-visit summary.

PROBLEM SOLVED Many prospective patients — especially out-of-town or international — won't travel for a first consult and are lost without a virtual option.

STACK OpenAI / Mistral (summaries) | Video SDK (Twilio Video / Daily) | Google Calendar

VALUE ★★★ Medium | **COMPLEXITY** Medium–High | **STATUS** Scaffolded only

9. Medical History Intake

`medical_history_intake`

DOES Runs a conversational intake that collects and structures medical history — allergies, medications, prior surgeries — before the visit, and flags contraindications.

PROBLEM SOLVED Paper/PDF intake forms are slow, often incomplete, and get re-typed by staff — pure wasted labor and a source of errors.

STACK OpenAI / Mistral | Postgres (JSONB)

VALUE ★★★★ High — large admin-time saver and a compliance win | **COMPLEXITY** Medium | **STATUS** Scaffolded only

10. Risk Assessment

`risk_assessment`

DOES Cross-references intake data, photo analysis, and procedure type to flag candidacy risks (BMI, medication interactions, smoking, etc.) for surgeon review before booking surgery.

PROBLEM SOLVED Missed risk factors lead to complications, last-minute cancellations, and real liability exposure for the practice.

STACK OpenAI GPT-4o

VALUE ★★★★ High — risk/liability reduction | **COMPLEXITY** High — must always route to a human clinician for the final call, never decide alone | **STATUS** Scaffolded only

11. Procedure Recommendation

`procedure_recommendation`

DOES Suggests relevant procedures to discuss based on the patient's stated goals and photos — framed as options for the surgeon conversation, never a prescription.

PROBLEM SOLVED Patients often don't know the correct name for what they want, which causes mismatched expectations and wasted consult time.

STACK OpenAI / Mistral

VALUE ★★★ Medium | **COMPLEXITY** Medium | **STATUS** Scaffolded only

12. Pre-Surgery Preparation

`pre_surgery_preparation`

DOES Delivers a personalized prep checklist and countdown reminders — medications to stop, fasting rules, what to bring — tailored to the specific procedure and surgery date.

PROBLEM SOLVED Patients arrive unprepared, causing day-of-surgery delays or outright cancellations.

STACK Mistral | WhatsApp / SMS / Email

VALUE ★★★★ High — protects the OR schedule | **COMPLEXITY** Low–Medium | **STATUS** Scaffolded only

Surgery Management — 6 agents — protecting the highest-value hours in the practice: the OR

13. Surgery Scheduling

`surgery_scheduling`

DOES Coordinates the full surgical calendar — matching patient, surgeon, OR, and anesthesia team availability into one confirmed slot.

PROBLEM SOLVED Multi-resource scheduling done by hand is slow and error-prone; a single conflict can cascade into a cancelled case.

STACK OpenAI / Mistral | Google Calendar | Postgres

VALUE ★★★★★ Flagship — OR time is the practice's highest-value resource | **COMPLEXITY** High | **STATUS** Scaffolded only

14. Surgeon Calendar

`surgeon_calendar`

DOES Keeps each surgeon's personal schedule — consults, surgeries, travel, blocked time — conflict-free and synced across systems.

PROBLEM SOLVED Surgeons frequently get double-booked across consult, surgery, and personal calendars when managed manually.

STACK Google Calendar | Mistral (light NLP)

VALUE ★★★ Medium | **COMPLEXITY** Medium | **STATUS** Scaffolded only

15. Operating Room Scheduler

`operating_room_scheduler`

DOES Optimizes OR utilization — sequencing procedures by duration and turnover time to minimize idle OR time between cases.

PROBLEM SOLVED Idle OR time is pure lost revenue; an OR can cost well over $1,000/hour to run whether or not it's in use.

STACK OpenAI (optimization reasoning) | Postgres

VALUE ★★★★★ Flagship — direct margin impact | **COMPLEXITY** High | **STATUS** Scaffolded only

16. Equipment Checklist

`equipment_checklist`

DOES Confirms every required tool, device, and sterilized instrument is ready before each procedure, flagging missing items to staff ahead of time.

PROBLEM SOLVED Missing equipment discovered day-of causes delays or outright case cancellations.

STACK Mistral | Postgres (inventory data)

VALUE ★★★ Medium — risk mitigation | **COMPLEXITY** Low–Medium | **STATUS** Scaffolded only

17. Implant Inventory

`implant_inventory`

DOES Tracks implant/device stock levels in real time and auto-generates reorder requests before stock runs low.

PROBLEM SOLVED Manual inventory tracking leads to stockouts that delay or reschedule surgeries.

STACK Mistral | Postgres | Email/SMS to suppliers

VALUE ★★★ Medium | **COMPLEXITY** Medium | **STATUS** Scaffolded only

18. Surgical Documentation

`surgical_documentation`

DOES Generates structured operative notes and surgical records from surgeon dictation or structured input, and files them to the patient's chart.

PROBLEM SOLVED Post-op documentation is time-consuming for surgeons and is often delayed or left incomplete.

STACK OpenAI GPT-4o | Deepgram (if dictated) | EHR service

VALUE ★★★★ High — major surgeon time saver + compliance | **COMPLEXITY** High | **STATUS** Scaffolded only

Post-Surgery Care — 6 agents — patient safety and satisfaction after the procedure

19. Recovery Follow-up

`recovery_followup`

DOES Checks in with patients at clinically appropriate intervals post-op (day 1, day 3, week 1...) via chat/WhatsApp, and escalates concerning responses to staff.

PROBLEM SOLVED Practices can't manually follow up with every patient at the right cadence — a missed early complication signal becomes a liability, and even a fine outcome feels neglected without check-ins.

STACK Mistral (routine) | OpenAI (escalation reasoning) | WhatsApp / SMS

VALUE ★★★★★ Flagship — patient safety, satisfaction, and the reviews that follow | **COMPLEXITY** Medium–High | **STATUS** Scaffolded only

20. Healing Progress Monitoring

`healing_monitoring`

DOES Collects patient-submitted recovery photos on a schedule, tracks visual healing progress over time, and flags anomalies for clinical review.

PROBLEM SOLVED Surgeons can't see how a patient is healing between visits — problems that photos would catch early often get caught late instead.

STACK OpenAI Vision | Cloudinary

VALUE ★★★★ High | **COMPLEXITY** High | **STATUS** Scaffolded only

21. Emergency Triage

`emergency_triage`

DOES Detects urgent/emergency language on any channel — chat, WhatsApp, and especially live voice calls — and immediately escalates to on-call staff, bypassing every queue.

PROBLEM SOLVED A missed emergency signal is the single highest-risk failure mode for any patient-facing AI system in a surgical practice.

STACK OpenAI GPT-4o (conservative, escalation-only) | Deepgram + Groq | Twilio (instant call/SMS to on-call staff)

VALUE ★★★★★ Flagship — safety-critical, non-negotiable to get right | **COMPLEXITY** High — needs the most rigorous testing of any agent in the whole system | **STATUS** Scaffolded only

22. Medication Reminder

`medication_reminder`

DOES Sends scheduled medication/dosage reminders post-op, tracks adherence, and alerts staff on repeated non-response.

PROBLEM SOLVED Non-adherence to post-op medication (antibiotics, pain management) is a preventable cause of complications.

STACK Mistral | WhatsApp / SMS

VALUE ★★★ Medium | **COMPLEXITY** Low | **STATUS** Scaffolded only

23. Wound Care Guidance

`wound_care_guidance`

DOES Delivers step-by-step aftercare instructions matched to the specific procedure and healing stage, and answers patient aftercare questions conversationally.

PROBLEM SOLVED Patients forget or misread printed aftercare sheets, leading to avoidable complications and panicked after-hours calls.

STACK OpenAI / Mistral (grounded in the practice's own protocols)

VALUE ★★★★ High | **COMPLEXITY** Medium | **STATUS** Scaffolded only

24. Recovery Progress Dashboard

`recovery_dashboard`

DOES A live, aggregated view for doctors and staff of every patient's recovery status, flags, and follow-up compliance across the whole practice — essentially the data engine behind the patient-timeline mockup in Section 06.

PROBLEM SOLVED There's no single place today to see "who needs attention right now" across dozens of recovering patients at once.

STACK Aggregates data already produced by other agents — Postgres — no new AI model required.

VALUE ★★★★ High — this is functionally the doctor panel's core screen | **COMPLEXITY** Medium (mostly frontend + aggregation logic) | **STATUS** Scaffolded only

Business & Operations — 7 agents — the money, the reputation, and the owner's view of it all

25. Cost Estimation

`cost_estimation`

DOES Generates instant, itemized procedure quotes based on procedure type, surgeon, and facility fees — shareable as a formal PDF quote.

PROBLEM SOLVED Price is the #1 question every prospective patient asks; when it's slow to get an answer, patients simply go elsewhere.

STACK Mistral (drafting) | Postgres (pricing data) | PDF generation

VALUE ★★★★★ Flagship — directly drives conversion | **COMPLEXITY** Medium | **STATUS** Scaffolded only

26. Payment & Invoice

`payment_invoice`

DOES Collects deposits and payments and sends invoices/receipts automatically, syncing with Stripe and supporting payment plans.

PROBLEM SOLVED Manual invoicing delays cash flow and creates ongoing admin overhead.

STACK Stripe | Mistral (message drafting) | SendGrid

VALUE ★★★★ High | **COMPLEXITY** Medium | **STATUS** Scaffolded only

27. Insurance Verification

`insurance_verification`

DOES Verifies insurance coverage and eligibility before the appointment where applicable (e.g. reconstructive procedures), flagging out-of-pocket costs upfront.

PROBLEM SOLVED Coverage surprises at the front desk cause cancellations and a bad first impression of the practice.

STACK OpenAI / Mistral | Insurance/EHR API integration

VALUE ★★★ Medium — most relevant to reconstructive/medical-necessity cases | **COMPLEXITY** High — payer integrations are the hardest part of this agent | **STATUS** Scaffolded only

28. Analytics Dashboard

`analytics_dashboard`

DOES Surfaces the metrics that actually grow the practice — revenue, conversion rate by agent/channel, no-show rate, lead sources, patient satisfaction — in one executive view.

PROBLEM SOLVED Owners fly blind on which channel or agent is genuinely driving revenue without this.

STACK BI/aggregation layer over Postgres, Mistral for natural-language insight summaries.

VALUE ★★★★★ Flagship — the owner's command center | **COMPLEXITY** Medium–High | **STATUS** Scaffolded only

29. Patient Feedback

`patient_feedback`

DOES Collects reviews and NPS automatically post-visit/post-recovery, routes negative feedback privately to staff before it becomes a public review, and nudges happy patients toward Google/social reviews.

PROBLEM SOLVED Practices either get too few reviews, or get blindsided by a negative one posted publicly with no warning.

STACK Mistral | SendGrid / WhatsApp

VALUE ★★★★ High — reputation is core to growth in this industry | **COMPLEXITY** Low–Medium | **STATUS** Scaffolded only

30. Marketing Follow-up

`marketing_followup`

DOES Re-engages every lead who didn't convert on first contact with the right message at the right time — educational content, testimonials, limited-time offers — across email, WhatsApp, and Instagram.

PROBLEM SOLVED Most leads don't book on the first touch; without a structured follow-up they're gone for good.

STACK Mistral (content) | OpenAI (personalization) | SendGrid / WhatsApp / Instagram

VALUE ★★★★ High | **COMPLEXITY** Medium | **STATUS** Scaffolded only

31. Lead Nurturing

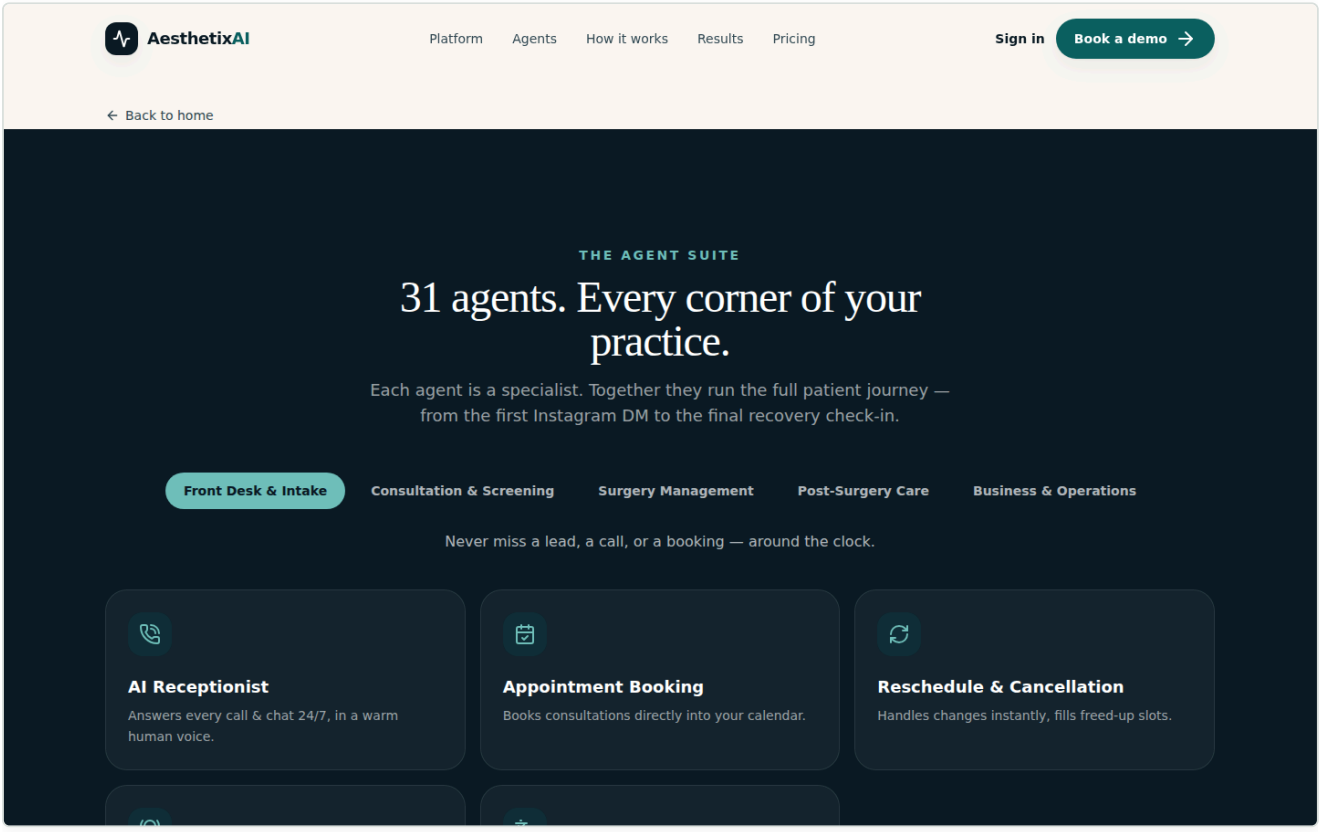
`lead_nurturing`

DOES Qualifies and warms inbound leads from every social channel — Instagram DMs, Facebook, WhatsApp, website chat — into booked consultations. This is the "one inbox, every channel" agent shown live on the Channels page.

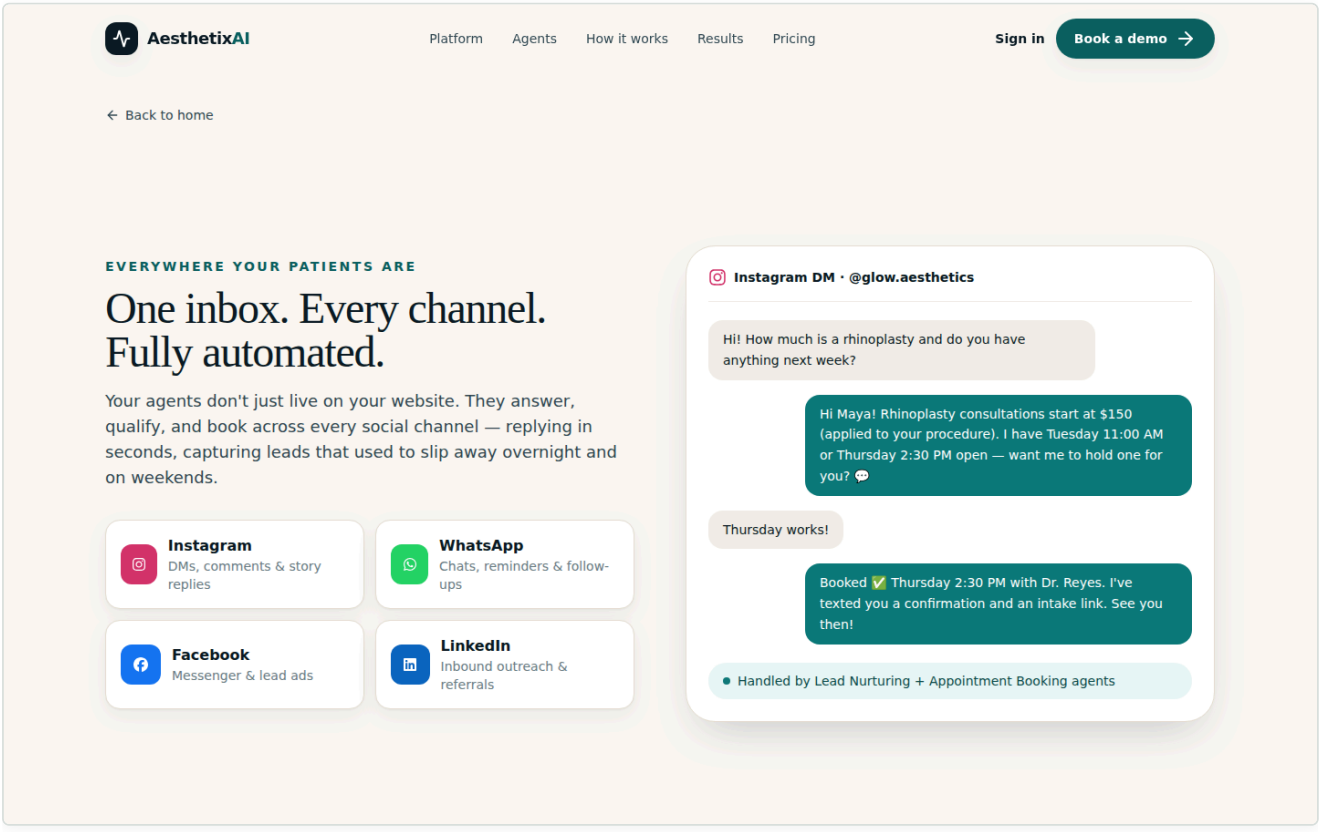
PROBLEM SOLVED Social DMs are where most aesthetic-medicine patients first reach out, and an unanswered DM is the single biggest silent revenue leak for a practice like this.

STACK OpenAI GPT-4o | Mistral (high-volume replies) | Instagram / Facebook / WhatsApp / LinkedIn

VALUE ★★★★★ Flagship | **COMPLEXITY** High | **STATUS** Scaffolded only



Live preview — the public Agent Suite catalog, filterable by these same five categories.



Live preview — the Channels page, showing the Instagram/WhatsApp/Facebook/LinkedIn omnichannel story behind Lead Nurturing and Appointment Booking.

SECTION 09

Pricing, Value & the Most Valuable Agents

Two ways to think about "pricing" here

First, the **product-level pricing** already coded into the live site — what a practice pays Aesthetixai per month:

PLAN	PRICE	WHAT'S INCLUDED
Solo	$690/mo	Front desk & intake agents, booking/reminders/rescheduling, 1 connected social channel, multilingual support, email support.
Practice	$1,690/mo	Everything in Solo + full consultation & surgery agents, post-surgery care/recovery suite, all social channels, analytics dashboard, priority onboarding.
Enterprise	Custom	All 31 agents fully configured, multi-location orchestration, custom integrations & EHR, BAA, dedicated success manager.

Second — and more useful for build prioritization — the **standalone value tier** of each agent, i.e. roughly how much measurable value it creates for a single practice per month if it were the only thing built. This is what should drive which agents Wahaj and Muneeb build first, independent of how the final packages get priced.

VALUE TIER	APPROX. MONTHLY VALUE	AGENTS IN THIS TIER
★★★★★ Flagship	$400–700 equivalent	Receptionist, Appointment Booking, AI Consultation, Surgery Scheduling, OR Scheduler, Emergency Triage, Recovery Follow-up, Cost Estimation, Analytics Dashboard, Lead Nurturing
★★★★ High	$250–400 equivalent	Reschedule & Cancellation, Appointment Reminder, Photo Analysis, Medical History Intake, Risk Assessment, Pre-Surgery Prep, Surgical Documentation, Healing Monitoring, Wound Care Guidance, Recovery Dashboard, Payment & Invoice, Patient Feedback, Marketing Follow-up
★★★ Medium	$150–250 equivalent	Multilingual Translation, Video Consultation, Procedure Recommendation, Surgeon Calendar, Equipment Checklist, Implant Inventory, Medication Reminder, Insurance Verification

The single most valuable agent: the AI Receptionist

If only one agent could be built first, it should be — and per the repo, already is — the **Receptionist**. It is the entry point for essentially every patient interaction (call, chat, WhatsApp), it is the flagship differentiator of the whole product ("Your practice, run by an AI workforce" — the marketing hero is literally built around a voice call), and it directly prevents the single most measurable revenue leak a practice has: a missed call that becomes a missed booking. It is also the only agent that already has real, working logic — everything else can be validated against the pattern it establishes.

Close behind it, forming what should be considered the **"never lose a lead" trio**: **Lead Nurturing** (catches everything the phone doesn't — Instagram/WhatsApp/Facebook DMs) and **Appointment Booking** (turns any of the above into an actual confirmed slot). Building these three first — Receptionist, Lead Nurturing, Appointment Booking — covers the entire top of the funnel and is the highest-leverage place to put the next two developers' time.

After that, **Cost Estimation** and **Recovery Follow-up** round out the top five: cost estimation because price is the #1 objection standing between an interested lead and a booked consult, and recovery follow-up because it's simultaneously a patient-safety agent and the biggest lever on the reviews that feed the whole marketing funnel back at the top.

SECTION 10

Team Assignment — Wahaj & Muneeb

Because the codebase is organized "one agent, one place," assigning an agent to a developer is literally just naming its slug — everything they need lives in three backend files and one frontend file, all named after that slug. No merge conflicts, no need to understand any other agent's code.

> **The exact checklist for whoever picks up an agent (e.g. `surgery_scheduling`):**
> 1. Open `backend/src/services/agents/surgery_scheduling_agent/surgery_scheduling_agent_services.py` — this is almost always the only file you change.
> 2. Replace the stub logic with the real behavior described in this document's Section 08 entry for your agent.
> 3. Only touch `surgery_scheduling_agent_controllers.py` if you're adding a brand-new operation, and `surgery_scheduling_agent_router.py` only if you're adding a new HTTP endpoint.
> 4. Update `frontend/src/data/agents/surgery_scheduling.ts` only if the public-facing name/description needs to change.
> 5. If your agent needs a new database table, add a model in `backend/src/models/` following the existing conventions and generate an Alembic migration.

Suggested first assignment (build order = value order from Section 09)

ORDER	AGENT	SUGGESTED OWNER	WHY FIRST
1	Appointment Booking	Wahaj	Completes the loop the Receptionist already starts — highest-leverage agent still fully unbuilt.
2	Lead Nurturing	Muneeb	Closes the social-channel gap; pairs naturally with the WhatsApp/Instagram service files that already exist.
3	Cost Estimation	Wahaj	Medium complexity, flagship value — a fast, visible win.
4	Recovery Follow-up	Muneeb	Patient-safety agent; establishes the WhatsApp check-in pattern that Medication Reminder and Wound Care Guidance can then reuse.
5+	Remaining agents, by value tier	Split by category	Once the top 5 are live, split the rest — e.g. one developer takes Surgery Management + Consultation, the other takes Post-Surgery Care + Business & Operations.

Emergency Triage is the one exception to "build in value order": because it's safety-critical, it should be scheduled early regardless of where it falls in the queue, and it deserves the most testing time of any agent before it touches a real patient.

SECTION 11

Roadmap & Next Steps

PHASE	FOCUS
Phase 1	Build out the top 5 flagship agents (Section 09) with real logic. Add the OpenAI/Mistral split and Deepgram+Groq voice pipeline for the Receptionist and Emergency Triage. Wire Clerk auth on the frontend.
Phase 2	Build the Doctor Panel against real data from Phase 1's agents, using the four design-reference mockups (Section 06) as the visual spec. Stand up the remaining High-value agents.
Phase 3	Build the Admin/Analytics panel once enough agents are producing real numbers to aggregate. Deploy to a VPS (2 vCPU/2GB to start), move SDK client construction to lazy-init, wire Postgres + Redis + Celery for background jobs.
Phase 4	Remaining Medium-tier agents, multi-location/Enterprise features, insurance/payer integrations, and scale the VPS to 4GB+ as traffic grows.

> *اگلا قدم: اوپر دیے گئے ٹاپ 5 ایجنٹس وہاج اور منیب کو اسائن کریں، اور ہر ایجنٹ کی سروس فائل میں اس دستاویز کے سیکشن 08 کی تفصیل کے مطابق اصل لاجک لکھنا شروع کریں۔*